

DBG-400 • THE CARD DROP • FINTECH / HARD

Ledger

A GST-ready, double-entry accounting system rebuilt from Bigcapital — where every invoice posts the right entries, every void reverses cleanly, and the trial balance never drifts.

DR	CR
1,00,000	1,00,000
18,000	18,000

Team: Cache Me If You Can · Original: github.com/bigcapitalhq/bigcapital · Commit fb21220

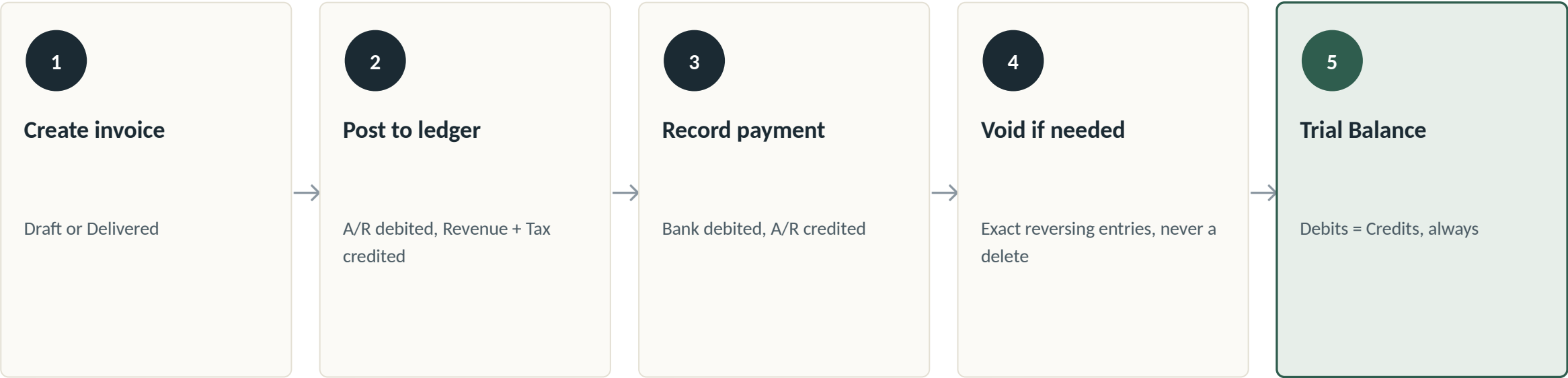
Rebuilt for the wholesale shop owner who raises GST invoices daily and whose books must always balance.

THE PROBLEM

Books that balance, even under pressure

For the **Business Owner** who struggles with mathematically inconsistent books and race conditions during high-volume invoicing, **Ledger** guarantees strict double-entry balancing and ACID-compliant concurrency locking — unlike the flat-file and lightweight tools that let silent data corruption slip through.

CORE FLOW



Seeded Chart of Accounts: 1000 Bank · 1100 A/R · 2000 A/P · 2100 Tax Payable · 3000 Equity · 4000 Sales Revenue · 5000 Expenses

One critical fix, one real differentiator



FIX

Unsafe concurrency, now row-locked

In Bigcapital: account balances were read and invoice numbers validated in application memory, sometimes outside the database transaction. No **SELECT ... FOR UPDATE** lock protected a concurrent write.

In Ledger: every balance-mutating read takes a pessimistic row lock inside an ACID transaction, and `invoice_no` / `accounts.code` carry true database UNIQUE constraints — not just an index.

Two staff recording a ₹10,000 payment at the same moment used to risk crediting Bank only once. Now it's physically impossible.



DIFFERENTIATOR

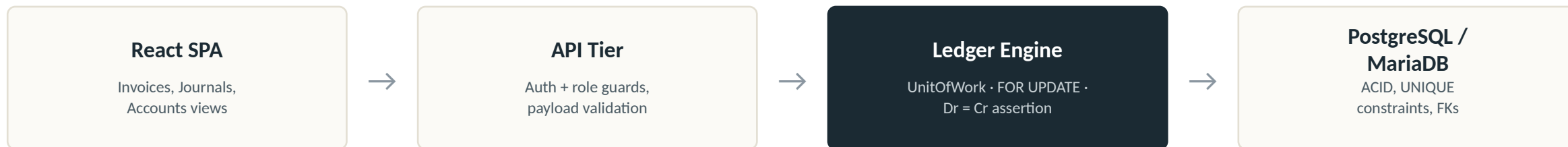
Duplicate / unusual payment alert

Bigcapital has no way to catch a staff member accidentally recording the same payment twice, or a payment wildly larger than a customer's usual amount — both common during fast daily entry, both invisible until reconciliation.

Ledger flags **a repeat amount for the same customer within a short window**, or **a payment 3× a customer's recent average** — for review, without blocking the transaction.

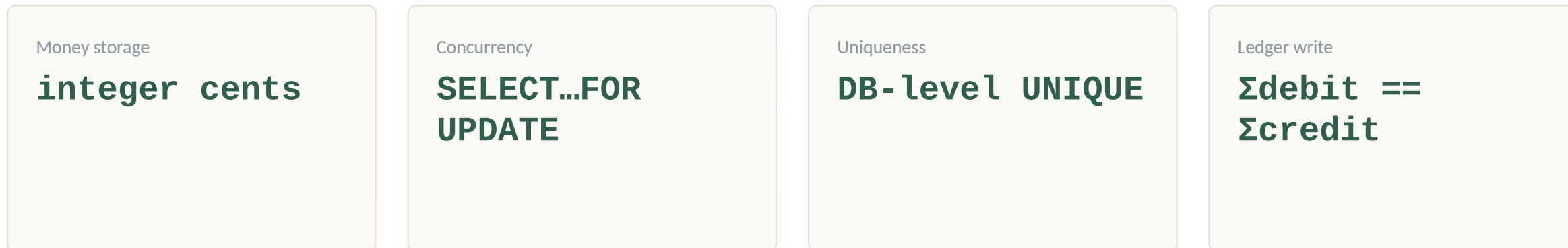
A duplicate entry corrupts a customer's balance silently — often not caught until they dispute their statement weeks later.

Architecture built for correctness, not just features



WHERE STATE LIVES

Everything persists in one relational database — no cache layer, no cached running balance. The Trial Balance is summed fresh from `accounts_transactions` on every request, so a cached figure can never drift from the real ledger.



PROOF, RUN LIVE

Three Killer Tests. Zero exceptions.



KT1

Every transaction balances

$\Sigma(\text{debits}) === \Sigma(\text{credits})$ asserted before any write — or the transaction never happens.



KT2

Voiding reverses cleanly

A void writes exact offsetting entries and sets status = Voided. Originals are never deleted, never mutated.



KT3

20 random ops, still balanced

Fire 20 concurrent invoice + payment operations; the Trial Balance holds to zero — row locks, proven.